Planejamento Lógico TaskTracker

Bootcamp II — Desafio Cumulativo | Entrega Inicial (Fase 1)

1. Apresentação e Entendimento do Problema

1.1 Identificação

Campo	Informação
Nome completo	Italo Nascimento de Sousa
Curso	Ciência de Dados e Machine Learning - CEUB
Disciplina	Bootcamp II (EAD - Disciplina Virtual)
Polo / Turma	Turma A - 0726 - Campus Virtual - Virtual - 2026/2
E-mail institucional	italo.thefato@sempreceub.com

1.2 O Problema a Ser Resolvido

Quem organiza as tarefas do dia sem nenhum sistema acaba distribuindo esses compromissos entre bilhetes de papel, mensagens enviadas para si mesmo, blocos de notas do celular e a própria memória. Cada um desses lugares guarda uma parte da informação, e nenhum deles guarda o conjunto completo.

Na prática, isso gera quatro dificuldades recorrentes:

- Esquecimento: uma tarefa anotada em um lugar que não é consultado com frequência simplesmente desaparece do radar até virar urgência.
- Falta de priorização: quando tudo está no mesmo nível, o critério de escolha passa a ser o que foi lembrado por último, e não o que é mais importante ou mais urgente.
- Ausência de visão do todo: sem uma lista única, a pessoa não consegue responder de forma rápida quantas tarefas ainda estão pendentes nem o que já foi concluído.
- Retrabalho e informação inconsistente: em equipe, o problema se multiplica, porque duas pessoas podem executar a mesma tarefa ou assumir que a outra já a executou.

O problema central, portanto, não é a falta de esforço de quem organiza, e sim a falta de um registro único, estruturado e consultável. É exatamente essa lacuna que o TaskTracker se propõe a preencher.

1.3 A Solução Proposta: TaskTracker

O TaskTracker é um sistema de gerenciamento de tarefas que centraliza, em um único lugar, o cadastro, a consulta e o acompanhamento das atividades do usuário. A aplicação será executada em terminal (interface de linha de comando) na Fase 2, escrita em Python, e manterá as tarefas em uma estrutura de dados na memória do programa durante a execução.

As funcionalidades previstas para esta primeira versão são:

- Cadastrar tarefa: registrar uma nova atividade com título, descrição, prioridade e data limite.
- Listar tarefas: exibir todas as tarefas registradas, com seu status e sua prioridade, permitindo enxergar o conjunto completo de uma só vez.
- Concluir tarefa: marcar uma tarefa como concluída, sem apagá-la, preservando o histórico do que já foi feito.
- Remover tarefa: excluir definitivamente uma tarefa cadastrada por engano ou que perdeu o sentido.
- Exibir resumo: informar o total de tarefas cadastradas, quantas estão pendentes e quantas já foram concluídas.

O ganho para o usuário é direto: em vez de reconstruir mentalmente o que precisa ser feito, ele consulta uma lista única, ordenada por prioridade, com o status de cada item explícito.

2. Decomposição do Problema e Dados do Sistema

2.1 Entradas de Dados

São as informações que o usuário digita no sistema no momento do cadastro de uma tarefa:

Entrada	Tipo	Obrigatório	Exemplo
Título	Texto	Sim	Entregar relatório de estatística
Descrição	Texto	Não	Revisar os gráficos antes de enviar
Prioridade	Texto (Alta / Média / Baixa)	Sim	Alta
Data limite	Data (DD/MM/AAAA)	Sim	30/09/2026
Opção do menu	Número inteiro (1 a 6)	Sim	1
Identificador da tarefa	Número inteiro	Sim (ao concluir/remover)	3

Além das entradas acima, o sistema gera automaticamente dois campos que o usuário não digita: o identificador único da tarefa (um número sequencial atribuído no cadastro) e o status inicial, que sempre começa como "Pendente".

2.2 Saídas de Dados

São as informações que o sistema devolve na tela para o usuário:

Saída	Descrição
Menu principal	Lista numerada das operações disponíveis, exibida a cada novo ciclo do programa.
Mensagem de confirmação	Aviso de que a tarefa foi cadastrada, concluída ou removida com sucesso.
Mensagem de erro	Aviso de que um dado obrigatório está vazio ou inválido, com a orientação para corrigir.
Lista de tarefas	Exibição de todas as tarefas com identificador, título, prioridade, data limite e status.
Aviso de lista vazia	Mensagem informando que ainda não existe nenhuma tarefa cadastrada.
Resumo quantitativo	Total de tarefas, quantidade de pendentes e quantidade de concluídas.

2.3 Regras do Sistema

São as restrições que o sistema deve respeitar independentemente do que o usuário digitar:

Código	Regra
RN01	O sistema não pode aceitar tarefas sem título. Se o campo estiver vazio, o cadastro é recusado e a solicitação é repetida.
RN02	A prioridade deve ser obrigatoriamente Alta, Média ou Baixa. Qualquer outro valor é rejeitado e solicitado novamente.
RN03	Toda tarefa recebe automaticamente um identificador único e sequencial, que nunca é reaproveitado, mesmo após remoções.
RN04	Toda tarefa nasce com o status "Pendente" e só muda para "Concluída" por ação explícita do usuário.
RN05	Não é possível concluir ou remover uma tarefa cujo identificador não exista na lista; o sistema avisa e retorna ao menu.
RN06	Uma tarefa já concluída não pode ser concluída novamente; o sistema informa que a operação não se aplica.
RN07	A data limite deve seguir o formato DD/MM/AAAA. Datas em formato inválido são recusadas.
RN08	Opções de menu fora do intervalo previsto são recusadas, e o menu é exibido novamente sem encerrar o programa.

3. Receita Lógica do Sistema (Algoritmo em Português)

Esta seção descreve, passo a passo e em linguagem comum, como o sistema funcionará. É a receita que será convertida em código Python na Fase 2.

3.1 Fluxo Geral do Programa

1. Iniciar o programa e criar uma lista vazia para armazenar as tarefas.
2. Criar um contador de identificadores começando em 1.
3. Exibir o menu com as opções: 1 - Cadastrar tarefa, 2 - Listar tarefas, 3 - Concluir tarefa, 4 - Remover tarefa, 5 - Ver resumo, 6 - Sair.
4. Pedir ao usuário que digite o número da opção desejada.
5. Se a opção não estiver entre 1 e 6, avisar que a opção é inválida e voltar ao passo 3.
6. Se a opção for válida, executar a rotina correspondente (descrita nos itens 3.2 a 3.5).
7. Ao terminar a rotina, voltar ao passo 3 e exibir o menu novamente.
8. Se a opção escolhida for 6, exibir uma mensagem de despedida e encerrar o programa.

3.2 Passo a Passo do Cadastro de Tarefa

1. Pedir ao usuário o título da tarefa.
2. Verificar se o título não está vazio. Se estiver, avisar que o título é obrigatório e voltar ao passo 1.
3. Pedir a descrição da tarefa, informando que o campo é opcional e pode ser deixado em branco.
4. Pedir a prioridade da tarefa (Alta, Média ou Baixa).
5. Verificar se a prioridade informada é uma das três opções válidas. Se não for, avisar e voltar ao passo 4.
6. Pedir a data limite no formato DD/MM/AAAA.
7. Verificar se a data está no formato correto. Se não estiver, avisar e voltar ao passo 6.
8. Atribuir à tarefa o valor atual do contador de identificadores e, em seguida, somar 1 ao contador.
9. Definir o status da tarefa como "Pendente".
10. Guardar a tarefa completa (identificador, título, descrição, prioridade, data limite e status) na lista de tarefas.
11. Exibir a mensagem de sucesso informando o identificador da tarefa recém-criada.
12. Retornar ao menu principal.

3.3 Passo a Passo da Visualização de Tarefas

1. Acessar a lista de tarefas.
2. Verificar se existe alguma tarefa cadastrada.
3. Se a lista estiver vazia, avisar o usuário que nenhuma tarefa foi cadastrada ainda e retornar ao menu.

4. Se houver tarefas, ordenar a lista por prioridade, exibindo primeiro as de prioridade Alta, depois Média e por fim Baixa.
5. Percorrer a lista ordenada e mostrar cada tarefa, uma por linha, com identificador, título, prioridade, data limite e status.
6. Após exibir todas as tarefas, mostrar o total de itens listados.
7. Retornar ao menu principal.

3.4 Passo a Passo da Conclusão de Tarefa

1. Exibir a lista de tarefas pendentes, seguindo a rotina de visualização.
2. Pedir ao usuário o identificador da tarefa que deseja concluir.
3. Procurar na lista uma tarefa com aquele identificador.
4. Se nenhuma tarefa for encontrada, avisar que o identificador não existe e retornar ao menu.
5. Se a tarefa for encontrada mas já estiver concluída, avisar o usuário e retornar ao menu.
6. Se a tarefa estiver pendente, alterar o seu status para "Concluída".
7. Exibir mensagem de sucesso com o título da tarefa concluída e retornar ao menu.

3.5 Passo a Passo da Remoção de Tarefa

1. Exibir a lista completa de tarefas.
2. Pedir ao usuário o identificador da tarefa que deseja remover.
3. Procurar na lista uma tarefa com aquele identificador.
4. Se nenhuma tarefa for encontrada, avisar que o identificador não existe e retornar ao menu.
5. Se a tarefa for encontrada, pedir uma confirmação explícita do usuário antes de excluir.
6. Se a remoção for confirmada, retirar a tarefa da lista e exibir mensagem de sucesso.
7. Se não for confirmada, cancelar a operação sem alterar a lista.
8. Retornar ao menu principal.

4. Mapeamento Arquitetural para Git e GitHub

4.1 Desenho da Árvore do Repositório

Estrutura de arquivos e pastas que será enviada ao GitHub na Fase 2:

```
tasktracker-projeto/
├── README.md                    # Apresentação do projeto no GitHub
├── .gitignore                   # Arquivos e pastas ignorados pelo Git
├── requirements.txt             # Bibliotecas usadas pelo projeto
├── docs/
│   └── planejamento_logico.pdf  # Este documento da Fase 1
└── src/
    └── main.py                  # Arquivo principal do programa em Python
```

A separação entre as pastas tem uma razão prática: docs/ guarda a documentação que explica as decisões do projeto, enquanto src/ guarda o código que efetivamente executa. Manter o código isolado em uma pasta própria evita que a raiz do repositório se misture com arquivos de apoio e

facilita o crescimento do projeto nas fases seguintes, quando o main.py provavelmente será dividido em módulos menores.

4.2 Plano de Aprendizado para Git e GitHub

Na Fase 2, espero consolidar o uso do Git como ferramenta de registro do histórico do projeto, e não apenas como um lugar para guardar arquivos. Especificamente, quero entender o que caracteriza um bom commit — o tamanho adequado de uma alteração e como escrever mensagens que descrevam a intenção da mudança, de forma que o histórico seja legível meses depois. Pretendo também praticar a criação de branches para desenvolver cada funcionalidade do TaskTracker separadamente antes de integrá-las à branch principal, aprender a resolver conflitos de código quando duas alterações tocam a mesma linha, e usar o .gitignore corretamente para manter fora do repositório arquivos temporários e de ambiente. Por fim, quero publicar o projeto no GitHub com um README claro, entendendo o fluxo completo entre o repositório local e o remoto, já que essa é a forma como o trabalho é organizado e revisado no ambiente profissional.

5. Link da Apresentação em Vídeo

https://youtu.be/n9jmv53A1Lc